Autorreferência e recursividade

Marco A L Barbosa
malbarbo.pro.br

Departamento de Informática
Universidade Estadual de Maringá



Este trabalho está licenciado com uma Licença Creative Commons - Atribuição-CompartilhaIgual 4.0 Internacional.
http://github.com/malbarbo/na-progfun

Introdução

Projete uma função que some uma sequência de números.

Como representar e processar uma quantidade de dados arbitrária?

Projete uma função que some uma sequência de números.

Como representar e processar uma quantidade de dados arbitrária?

- Vamos criar tipos de dados com autorreferência, isto é, tipos de dados que são definidos em termos deles mesmos
- Vamos usar funções recursivas para processar dados com autorreferência

Listas

O tipo de dado com autorreferência mais comum nas linguagens funcionais é a lista.

Vamos tentar criar uma definição para lista de números.

A ideia é criar uma estrutura com dois campos. O primeiro campo representa o primeiro item na lista e o segundo campo representa o restante da lista (que é uma lista).

```
(struct lista (primeiro resto) #:transparent)
;; Representa uma lista de números
;;  primeiro: Número - primeiro elemento da lista
;;  resto:    Lista  - restante da lista
```

Utilizando esta definição, vamos tentar representar uma lista com os valores 4, 2 e 8.

```
(define ldn (lista 4 (lista 2 (lista 8 ???))))
```

O problema com esta definição é que ela não tem fim. Uma lista é definida em termos de outra lista, que é definida em termos de outra lista, etc.

Precisamos de uma maneira de criar uma lista diretamente, que não seja em termos de outra lista. Que lista pode ser essa?

Precisamos de uma maneira de criar uma lista diretamente, que não seja em termos de outra lista. Que lista pode ser essa?

A lista vazia.

Listas

Uma **ListaDeNúmeros** é um dos valores:

Listas

Uma **ListaDeNúmeros** é um dos valores:

- `(vazia)`; ou
- `(link Número ListaDeNúmeros)`, onde link é uma estrutura com dois campos: `primeiro` e `resto`

Uma **ListaDeNúmeros** é um dos valores:

- `(vazia)`; ou
- `(link Número ListaDeNúmeros)`, onde link é uma estrutura com dois campos: `primeiro` e `resto`

Definição no Racket

```
(struct vazia () #:transparent)

(struct link (primeiro resto) #:transparent)
;; Uma ListaDeNúmeros é um dos valores
;;   - (vazia); ou
;;   - (link Numero ListaDeNúmeros)
```

Listas

```
> (define ldn1 (link 3 (vazia)))          ; Lista com o 3
> (define ldn2 (link 8 (link 7 (vazia)))) ; Lista com o 8 e 7
> ldn1
(link 3 (vazia))
> ldn2
(link 8 (link 7 (vazia)))
> (link-primeiro ldn2)
8
> (link-resto ldn2)
(link 7 (vazia))
> (link-resto ldn1)
(vazia)
> (link-primeiro (link-resto ldn1))
. . link-primeiro: contract violation
  expected: link?
  given: (vazia)
```

```
;; Lista com os elementos 8 e 7
> (define ldn2 (link 8 (link 7 (vazia))))
;; Define uma lista a partir de uma lista existente
> (define ldn3 (link 4 ldn2))
> ldn3
(link 4 (link 8 (link 7 (vazia))))
> (link-primeiro ldn3)
4
> (link-resto ldn3)
(link 8 (link 7 (vazia)))
> (link-primeiro (link-resto ldn3))
8
```

Nós vimos anteriormente que a forma do dado de entrada de uma função sugere uma forma para o corpo da função.

Qual é a forma para o corpo de uma função que o tipo da entrada ListaDeNúmeros sugere?

Uma condicional com dois casos:

- A lista é vazia
- A lista é um link

Uma condicional com dois casos:

- A lista é vazia
- A lista é um link

Em Racket

```
(define (fn-para-ldn ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (link-resto ldn))]))
```

Qual é o tipo do resultado de `(link-primeiro ldn)`?

Qual é o tipo do resultado de `(link-primeiro ldn)`? Um número, que é um valor atômico.

Qual é o tipo do resultado de `(link-resto ldn)`?

Qual é o tipo do resultado de `(link-primeiro ldn)`? Um número, que é um valor atômico.

Qual é o tipo do resultado de `(link-resto ldn)`? Uma lista, que é uma união.

Um valor atômico pode ser processado diretamente, mas como processar uma lista?

Qual é o tipo do resultado de `(link-primeiro ldn)`? Um número, que é um valor atômico.

Qual é o tipo do resultado de `(link-resto ldn)`? Uma lista, que é uma união.

Um valor atômico pode ser processado diretamente, mas como processar uma lista? Fazendo análise dos casos.

Vamos fazer uma alteração no modelo `fn-para-ldn` e adicionar uma chamada recursiva para processar `(link-resto ldn)`. Essa alteração pode parecer meio "mágica" agora, mas ela vai ficar mais clara em breve.

Uma ListaDeNúmeros é um dos valores:

- `(vazia)`; ou
- `(link Número ListaDeNúmeros)`, onde link é uma estrutura com dois campos: `primeiro` e `resto`

Uma ListaDeNúmeros é um dos valores:

- `(vazia)`; ou
- `(link Número ListaDeNúmeros)`, onde link é uma estrutura com dois campos: `primeiro` e `resto`

Modelo para funções que a entrada é ListaDeNúmeros

```
(define (fn-para-ldn ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (fn-para-ldn (link-resto ldn)))]))
```

Uma ListaDeNúmeros é um dos valores:

- `(vazia)`; ou
- `(link Número ListaDeNúmeros)`, onde link é uma estrutura com dois campos: `primeiro` e `resto`

Modelo para funções que a entrada é ListaDeNúmeros

```
(define (fn-para-ldn ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (fn-para-ldn (link-resto ldn)))]))
```

Quais são as relações entre a definição de `ListaDeNúmeros` e o modelo?

Uma ListaDeNúmeros é um dos valores:

- `(vazia)`; ou
- `(link Número ListaDeNúmeros)`, onde link é uma estrutura com dois campos: `primeiro` e `resto`

Modelo para funções que a entrada é ListaDeNúmeros

```
(define (fn-para-ldn ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (fn-para-ldn (link-resto ldn)))]))
```

Quais são as relações entre a definição de `ListaDeNúmeros` e o modelo?

- A definição tem dois casos, o modelo também
- A autorreferência na definição do dado sugere uma chamada recursiva no modelo

Exemplo: soma

Defina uma função que some os valores de uma lista de números.

Exemplo: soma

Especificação

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))

(define (soma ldn) 0)
```

Exemplo: soma

Especificação

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))

(define (soma ldn) 0)
```

E agora, como escrevemos a implementação?

Exemplo: soma

Especificação

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))

(define (soma ldn) 0)
```

E agora, como escrevemos a implementação? Vamos partir do modelo que criamos!

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))


(define (fn-para-ldn ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (fn-para-ldn (link-resto ldn)))]))
```

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

O que fazemos agora?

```
(define (fn-para-ldn ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (fn-para-ldn (link-resto ldn)))]))
```

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (fn-para-ldn ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (fn-para-ldn (link-resto ldn)))]))
```

O que fazemos agora?

Mudamos o nome da função tanto na definição quanto na chamada recursiva.

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))


(define (soma ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

O que fazemos agora?

```
(define (soma ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

O que fazemos agora?

Vamos preencher as lacunas. Qual deve ser o resultado quando a lista é vazia?

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) ...]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

O que fazemos agora?

Vamos preencher as lacunas. Qual deve ser o resultado quando a lista é vazia? 0.

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))


(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

O que fazemos agora?

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

O que fazemos agora?

Analisamos o caso em que a lista não é vazia. O modelo está sugerindo fazer a chamada recursiva para o resto da lista.

Aqui vem o ponto crucial! Mesmo a função não estando completa, nós vamos assumir que ela produz a resposta correta para o resto da lista.

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 2 queremos obter a soma de `(link 3 (vazia))` que é 3. O que temos para compor o resultado?

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 2 queremos obter a soma de `(link 3 (vazia))` que é 3. O que temos para compor o resultado?

- 3, que é `(link-primeiro ldn)`
- 0, que é `(soma (link-resto ldn))`

Como obtemos o resultado que queremos?

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 2 queremos obter a soma de `(link 3 (vazia))` que é 3. O que temos para compor o resultado?

- 3, que é `(link-primeiro ldn)`
- 0, que é `(soma (link-resto ldn))`

Como obtemos o resultado que queremos?
`(+ 3 0)`

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 3 queremos obter a soma de `(link 2 (link 5 (vazia)))` que é 7. O que temos para compor o resultado?

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 3 queremos obter a soma de `(link 2 (link 5 (vazia)))` que é 7. O que temos para compor o resultado?

- 2, que é `(link-primeiro ldn)`
- 5, que é `(soma (link-resto ldn))`

Como obtemos o resultado que queremos?

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 3 queremos obter a soma de `(link 2 (link 5 (vazia)))` que é 7. O que temos para compor o resultado?

- 2, que é `(link-primeiro ldn)`
- 5, que é `(soma (link-resto ldn))`

Como obtemos o resultado que queremos?
`(+ 2 5)`

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 4 queremos obter a soma de `(link 4 (link 1 (link 3 (vazia))))` que é 8. O que temos para compor o resultado?

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 4 queremos obter a soma de `(link 4 (link 1 (link 3 (vazia))))` que é 8. O que temos para compor o resultado?

- 4, que é `(link-primeiro ldn)`
- 4, que é `(soma (link-resto ldn))`

Como obtemos o resultado que queremos?

Exemplo: soma

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))
```

```
(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (... (link-primeiro ldn)
          (soma (link-resto ldn)))]))
```

No exemplo 4 queremos obter a soma de `(link 4 (link 1 (link 3 (vazia))))` que é 8. O que temos para compor o resultado?

- 4, que é `(link-primeiro ldn)`
- 4, que é `(soma (link-resto ldn))`

Como obtemos o resultado que queremos?
`(+ 4 4)`

Exemplo: soma

Agora que compreendemos como o resultado é formado, podemos completar o corpo da função.

Exemplo: soma

Agora que compreendemos como o resultado é formado, podemos completar o corpo da função.

```
;; ListaDeNúmeros -> Número
;; Soma os valores de ldn.
(examples
 (check-equal? (soma (vazia)) 0)
 (check-equal? (soma (link 3 (vazia))) 3) ; (+ 3 0)
 (check-equal? (soma (link 2 (link 5 (vazia)))) 7)) ; (+ 2 (+ 5 0))
 (check-equal? (soma (link 4 (link 1 (link 3 (vazia))))) 8)) ; (+ 4 (+ 1 (+ 3 0)))

(define (soma ldn)
  (cond
    [(vazia? ldn) 0]
    [else
     (+ (link-primeiro ldn)
        (soma (link-resto ldn)))]))
```

Verificação: Ok.

Revisão: Ok.

O Racket já vem com listas pré-definidas

- `empty` ao invés de `(vazia)`
- `cons` ao invés de `link`
- `first` ao invés de `link-primeiro`
- `rest` ao invés de `link-resto`

O Racket já vem com listas pré-definidas

- `empty` ao invés de `(vazia)`
- `cons` ao invés de `link`
- `first` ao invés de `link-primeiro`
- `rest` ao invés de `link-resto`

Outras funções (os propósitos são aproximados)

- `empty?` verifica se uma lista é vazia
- `cons?` verifica se uma lista não é vazia
- `list?` verifica se um valor é uma lista

Uma **ListaDeNúmeros** é um dos valores

- `empty`; ou
- `(cons Número ListaDeNúmeros)`

```
;; Modelo de funções para ListaDeNúmeros
(define (fn-para-ldn ldn)
  (cond
    [(empty? ldn) ...]
    [else
      ... (first ldn)
      ... (fn-para-ldn (rest ldn)) ... ]))
```

Listas

```
> (define ldn1 (cons 3 empty)) ; Lista com o elemento 3
> (define ldn2 (cons 8 (cons 7 empty))) ; Lista com 8 e 7
> ldn1
'(3)
> ldn2
'(8 7)
> (first ldn2)
8
> (rest ldn2)
'(7)
> (rest (rest ldn2))
'()
> (first (rest ldn1))
. . first: contract violation
  expected: (and/c list? (not/c empty?))
    given: '()
```

```
;; Lista com os elementos 8 e 7
> (define ldn2 (cons 8 (cons 7 empty)))
;; Defini uma lista a partir de uma lista existente
> (define ldn3 (cons 4 ldn2))
> ldn3
'(4 8 7)
> (first ldn3)
4
> (rest ldn3)
'(8 7)
> (first (rest ldn3))
8
```

O Racket oferece uma forma conveniente de criar listas

```
> (list 4 5 6 -2 20)
'(4 5 6 -2 20)
```

Em geral

```
(list <a1> <a2> ... <an>)
```

é equivalente a

```
(cons <a1>
      (cons <a2>
            (cons ...
                  (cons <an> empty) ...)))
```

Exemplo: contém

Defina uma função que verifique se um dado valor está em uma lista de números.

Exemplo: contém

```
;; ListaDeNúmeros Número -> Booleano
;; Produz #t se v está em ldn; #f caso contrário.
(examples
 (check-equal? (contem? empty 3) #f)
 (check-equal? (contem? (cons 3 empty) 3) #t)
 (check-equal? (contem? (cons 3 empty) 4) #f)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 4) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 10) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 8) #f))
```

Iniciamos com a especificação.

```
(define (contem? ldn v) #f)
```

Exemplo: contém

```
;; ListaDeNúmeros Número -> Booleano
;; Produz #t se v está em ldn; #f caso contrário.
(examples
 (check-equal? (contem? empty 3) #f)
 (check-equal? (contem? (cons 3 empty) 3) #t)
 (check-equal? (contem? (cons 3 empty) 4) #f)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 4) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 10) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 8) #f))
```

Para implementação partimos do modelo.

```
(define (fn-para-ldn ldn)
  (cond
    [(empty? ldn) ...]
    [else
      ... (first ldn)
      ... (fn-para-ldn (rest ldn)) ... ]))
```

Exemplo: contém

```
;; ListaDeNúmeros Número -> Booleano
;; Produz #t se v está em ldn; #f caso contrário.
(examples
 (check-equal? (contem? empty 3) #f)
 (check-equal? (contem? (cons 3 empty) 3) #t)
 (check-equal? (contem? (cons 3 empty) 4) #f)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 4) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 10) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 8) #f))
```

```
(define (contem? ldn v)
  (cond
    [(empty? ldn) ... v]
    [else
      ... (first ldn)
      ... v
      ... (contem? (rest ldn) v) ... ]))
```

Em seguida ajeitamos o nome da função e adicionamos o parâmetro `v` na definição, na chamada recursiva e como valor disponível nos dois casos.

Exemplo: contém

```
;; ListaDeNúmeros Número -> Booleano
;; Produz #t se v está em ldn; #f caso contrário
(examples
 (check-equal? (contem? empty 3) #f)
 (check-equal? (contem? (cons 3 empty) 3) #t)
 (check-equal? (contem? (cons 3 empty) 4) #f)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 4) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 10) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 8) #f))
```

```
(define (contem? ldn v)
  (cond
    [(empty? ldn) #f]
    [else
      ... (first ldn)
      ... v
      ... (contem? (rest ldn) v) ... ]))
```

Analisando os exemplos definimos o caso em que a lista é vazia.

Exemplo: contém

```
;; ListaDeNúmeros Número -> Booleano
;; Produz #t se v está em ldn; #f caso contrário
(examples
 (check-equal? (contem? empty 3) #f)
 (check-equal? (contem? (cons 3 empty) 3) #t)
 (check-equal? (contem? (cons 3 empty) 4) #f)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 4) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 10) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 8) #f))
```

```
(define (contem? ldn v)
  (cond
    [(empty? ldn) #f]
    [else
      ... (first ldn)
      ... v
      ... (contem? (rest ldn) v) ... ]))
```

Agora analisamos o caso em que a `ldn` não é vazia. Temos `(first ldn)`, `v` e `(contem? (rest ldn) v)` (se o resto de `ldn` contém `v`). Como combinar esses elementos para determinar `(contem? ldn v)` (se `ldn` contém `v`)?

Exemplo: contém

```
;; ListaDeNúmeros Número -> Booleano
;; Produz #t se v está em ldn; #f caso contrário
(examples
 (check-equal? (contem? empty 3) #f)
 (check-equal? (contem? (cons 3 empty) 3) #t)
 (check-equal? (contem? (cons 3 empty) 4) #f)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 4) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 10) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 8) #f))
```

```
(define (contem? ldn v)
  (cond
    [(empty? ldn) #f]
    [else
     (if (= v (first ldn))
         #t
         (contem? (rest ldn) v))]))
```

Quando `(first ldn)` é igual a `v`, podemos gerar a resposta `#t` diretamente, independente da resposta da chamada recursiva. Caso contrário, a resposta se `ldn` contém `v` e a mesma se `(rest ldn)` contém `v`, ou seja, a resposta para `(contem? ldn v)` é a equivalente a `(contem? (rest ldn) v)`.

Exemplo: contém

```
;; ListaDeNúmeros Número -> Booleano
;; Produz #t se v está em ldn; #f caso contrário
(examples
 (check-equal? (contem? empty 3) #f)
 (check-equal? (contem? (cons 3 empty) 3) #t)
 (check-equal? (contem? (cons 3 empty) 4) #f)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 4) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 10) #t)
 (check-equal? (contem? (cons 4 (cons 10 (cons 3 empty))) 8) #f))
```

Revisão.

```
(define (contem? ldn v)
  (cond
    [(empty? ldn) #f]
    [else
     (or (= v (first ldn))
         (contem? (rest ldn) v))]))
```

Exemplo: remove negativos

Defina uma função que remova todos os número negativos de uma lista de números.

Exemplo: remove negativos

```
;; ListaDeNúmeros -> ListaDeNúmeros
;; Produz uma nova lista removendo os valores negativos de ldn.
(examples
 (check-equal? (remove-negativos empty) empty)
 (check-equal? (remove-negativos (cons -1 (cons 2 (cons -3 empty))))
               (cons 2 empty))
 (check-equal? (remove-negativos (cons 3 (cons 4 (cons -2 empty))))
               (cons 3 (cons 4 empty))))

(define (remove-negativos ldn) empty)
```

Iniciamos com a especificação.

Exemplo: remove negativos

```
;; ListaDeNúmeros -> ListaDeNúmeros
;; Produz uma nova lista removendo os valores negativos de ldn.
(examples
 (check-equal? (remove-negativos empty) empty)
 (check-equal? (remove-negativos (cons -1 (cons 2 (cons -3 empty))))
               (cons 2 empty))
 (check-equal? (remove-negativos (cons 3 (cons 4 (cons -2 empty))))
               (cons 3 (cons 4 empty))))

(define (remove-negativos ldn)
  (cond
    [(empty? ldn) ...]
    [else
      ... (first ldn)
      ... (remove-negativos (rest ldn)) ... ]))
```

Para implementação partimos do modelo e ajustamos o nome.

Exemplo: remove negativos

```
;; ListaDeNúmeros -> ListaDeNúmeros
;; Produz uma nova lista removendo os valores negativos de ldn.
(examples
 (check-equal? (remove-negativos empty) empty)
 (check-equal? (remove-negativos (cons -1 (cons 2 (cons -3 empty))))
               (cons 2 empty))
 (check-equal? (remove-negativos (cons 3 (cons 4 (cons -2 empty))))
               (cons 3 (cons 4 empty))))

(define (remove-negativos ldn)
  (cond
    [(empty? ldn) empty]
    [else
      ... (first ldn)
      ... (remove-negativos (rest ldn)) ... ]))
```

Analisando os exemplos definimos o caso em que a lista é vazia.

Exemplo: remove negativos

```
;; ListaDeNúmeros -> ListaDeNúmeros
;; Produz uma nova lista removendo os valores negativos de ldn.
(examples
 (check-equal? (remove-negativos empty) empty)
 (check-equal? (remove-negativos (cons -1 (cons 2 (cons -3 empty))))
               (cons 2 empty))
 (check-equal? (remove-negativos (cons 3 (cons 4 (cons -2 empty))))
               (cons 3 (cons 4 empty))))

(define (remove-negativos ldn)
  (cond
    [(empty? ldn) empty]
    [else
     (if (< (first ldn) 0)
         (remove-negativos (rest ldn))
         (cons (first ldn)
               (remove-negativos (rest ldn))))]))
```

Analisando os exemplos definimos o caso em que a lista não é vazia.

Verificação: Ok.

Revisão: exercício.

Exemplo: soma x

Defina uma função que soma um valor x em cada elemento de uma lista de números.

Exemplo: soma x

```
;; ListaDeNúmeros Número -> ListaDeNúmeros
;; Produz uma nova lista somando x a cada elemento de ldn.
(examples
 (check-equal? (soma-x empty 4)
               empty)
 (check-equal? (soma-x (cons 4 (cons 2 empty)) 5)
               (cons 9 (cons 7 empty)))
 (check-equal? (soma-x (cons 3 (cons -1 (cons 4 empty))) -2)
               (cons 1 (cons -3 (cons 2 empty)))))

(define (soma-x ldn x) empty)
```

Iniciamos com a especificação.

Exemplo: soma x

```
;; ListaDeNúmeros Número -> ListaDeNúmeros
;; Produz uma nova lista somando x a cada elemento de ldn.
(examples
 (check-equal? (soma-x empty 4)
               empty)
 (check-equal? (soma-x (cons 4 (cons 2 empty)) 5)
               (cons 9 (cons 7 empty)))
 (check-equal? (soma-x (cons 3 (cons -1 (cons 4 empty))) -2)
               (cons 1 (cons -3 (cons 2 empty)))))

(define (soma-x ldn x)
  (cond
    [(empty? ldn) ... x]
    [else
      ... (first ldn)
      ... x
      ... (soma-x (rest ldn) x) ... ]))
```

Para implementação partimos do modelo e ajustamos o nome e adicionamos o parâmetro x.

Exemplo: soma x

```
;; ListaDeNúmeros Número -> ListaDeNúmeros
;; Produz uma nova lista somando x a cada elemento de ldn.
(examples
 (check-equal? (soma-x empty 4)
               empty)
 (check-equal? (soma-x (cons 4 (cons 2 empty)) 5)
               (cons 9 (cons 7 empty)))
 (check-equal? (soma-x (cons 3 (cons -1 (cons 4 empty))) -2)
               (cons 1 (cons -3 (cons 2 empty)))))

(define (soma-x ldn x)
  (cond
    [(empty? ldn) empty]
    [else
      ... (first ldn)
      ... x
      ... (soma-x (rest ldn) x) ... ]))
```

Analisando os exemplos definimos o caso em que a lista é vazia.

Exemplo: soma x

```
;; ListaDeNúmeros Número -> ListaDeNúmeros
;; Produz uma nova lista somando x a cada elemento de ldn.
(examples
 (check-equal? (soma-x empty 4)
               empty)
 (check-equal? (soma-x (cons 4 (cons 2 empty)) 5)
               (cons 9 (cons 7 empty)))
 (check-equal? (soma-x (cons 3 (cons -1 (cons 4 empty))) -2)
               (cons 1 (cons -3 (cons 2 empty)))))

(define (soma-x ldn x)
  (cond
    [(empty? ldn) empty]
    [else
     (cons (+ x (first ldn))
           (soma-x (rest ldn) x))]))
```

Analisando os exemplos definimos o caso em que a lista não é vazia.

Verificação: Ok.

Revisão: Ok.

Exemplos: junta com “,” e “e”

Projete uma função que junte todos os elementos de uma lista de strings (não vazias) separando-os com `", "` ou/e `" e "`, de acordo com a gramática do Português.

Exemplos: junta com “,” e “e”

```
;; ListaDeStrings -> String
;; Parece difícil escrever o propósito...
;; Vamos fazer os exemplos primeiro.
(define (junta-virgula-e lst) "")
```

Exemplos

Exemplos: junta com “,” e “e”

```
;; ListaDeStrings -> String
;; Parece difícil escrever o propósito...
;; Vamos fazer os exemplos primeiro.
(define (junta-virgula-e lst) "")
```

Exemplos

```
(junta-virgula-e empty) → ""

(junta-virgula-e (list "maça")) → "maça"

(junta-virgula-e (list "banana" "maça")) → "banana e maça"

(junta-virgula-e (list "mamão" "banana" "maça")) → "mamão, banana e maça"

(junta-virgula-e (list "aveia" "mamão" "banana" "maça")) → "aveia, mamão, banana e maça"
```

Em todos os exemplos as repostas são calculadas da mesma forma?

Exemplos: junta com “,” e “e”

```
;; ListaDeStrings -> String
;; Parece difícil escrever o propósito...
;; Vamos fazer os exemplos primeiro.
(define (junta-virgula-e lst) "")
```

Exemplos

```
(junta-virgula-e empty) → ""

(junta-virgula-e (list "maça")) → "maça"

(junta-virgula-e (list "banana" "maça")) → "banana e maça"

(junta-virgula-e (list "mamão" "banana" "maça")) → "mamão, banana e maça"

(junta-virgula-e (list "aveia" "mamão" "banana" "maça")) → "aveia, mamão, banana e maça"
```

Em todos os exemplos as repostas são calculadas da mesma forma? Não! Os três primeiros exemplos tem uma forma especifica, que não é recursiva.

Então precisamos criar três casos base

Exemplos: junta com “,” e “e”

```
;; ListaDeStrings -> String
;; Se a lista é vazia, devolve "".
;; Se a lista tem apenas um elemento, devolve
;; esse elemento.
;; Senão, junta as strings de lst, separando-as
;; com ", ", com exceção da última string, que
;; é separada com " e ".
(examples
  (check-equal? (junta-virgula-e empty) "")
  (check-equal?
    (junta-virgula-e (list "maça")) "maça")
  (check-equal?
    (junta-virgula-e (list "mamão" "banana"
                           "maça"))
    "mamão, banana e maça")
  (check-equal?
    (junta-virgula-e (list "aveia" "mamão"
                           "banana" "maça"))
    "aveia, mamão, banana e maça"))
(define (junta-virgula-e lst) "")
```

Exemplos: junta com “,” e “e”

```
;; ListaDeStrings -> String
;; Se a lista é vazia, devolve "".
;; Se a lista tem apenas um elemento, devolve
;; esse elemento.
;; Senão, junta as strings de lst, separando-as
;; com ", ", com exceção da última string, que
;; é separada com " e ".
(examples
  (check-equal? (junta-virgula-e empty) "")
  (check-equal?
    (junta-virgula-e (list "maça")) "maça")
  (check-equal?
    (junta-virgula-e (list "mamão" "banana"
                           "maça"))
    "mamão, banana e maça")
  (check-equal?
    (junta-virgula-e (list "aveia" "mamão"
                           "banana" "maça"))
    "aveia, mamão, banana e maça"))
(define (junta-virgula-e lst) "")
```

Implementação

```
(define (junta-virgula-e lst)
  (cond
    [(empty? lst) ""]
    [(empty? (rest lst)) (first lst)]
    [(empty? (rest (rest lst)))
     (string-append (first lst)
                    " e "
                    (second lst))]
    [else
      (string-append (first lst)
                     ", "
                     (junta-virgula-e (rest lst)))]))
```

Números Naturais

Um número natural é atômico ou composto?

Um número natural é atômico ou composto?

- Atômico quando usado em operações aritméticas, comparações, etc
- Composto quando uma iteração precisa ser feita baseado no valor do número

Se um número natural pode ser visto como dado composto

- Quais são as partes que compõe o número?
- Como (de)compor um número?

Definição

Um número **Natural** é

- 0; ou
- `(add1 n)` onde *n* é um número **Natural**

Baseado nesta definição, criamos um modelo para funções com números naturais

Um número **Natural** é

- 0; ou
- `(add1 n)` onde *n* é um número **Natural**

Baseado nesta definição, criamos um modelo para funções com números naturais

```
(define (fn-para-natural n)
  (cond
    [(zero? n) ...]
    [else
      (... n
           (fn-para-natural (sub1 n)))]))
```

Definição

```
;; as funções add1, sub1 e zero? são pré-definidas

;; compõe um novo natural a partir de um existente
;; semelhante ao cons
> (add1 8)
9
;; decompõe um natural
;; semelhante a rest
> (sub1 8)
7
;; verifica se um natural é 0
;; semelhante ao empty?
> (zero? 8)
#f
> (zero? 0)
#t
```

Exemplo: soma naturais

Dado um número natural n, defina uma função que some os números naturais menores ou iguais a n.

Exemplo: soma naturais

Especificação

```
;; Natural -> Natural
;; Soma todos os números naturais de 0 até n
(examples
 (check-equal? (soma-nat 0) 0)
 (check-equal? (soma-nat 1) 1)  ; (+ 1 0)
 (check-equal? (soma-nat 2) 3)  ; (+ 2 1 0)
 (check-equal? (soma-nat 3) 6)) ; (+ 3 2 1 0)
(define (soma-nat n) 0)
```

Exemplo: soma naturais

Implementação: a partir do modelo.

```
;; Natural -> Natural
;; Soma todos os números naturais de 0 até n
(examples
 (check-equal? (soma-nat 0) 0)
 (check-equal? (soma-nat 1) 1)  ; (+ 1 0)
 (check-equal? (soma-nat 2) 3)  ; (+ 2 1 0)
 (check-equal? (soma-nat 3) 6)) ; (+ 3 2 1 0)
(define (soma-nat n)
  (cond
    [(zero? n) ...]
    [else
      (... n
           (soma-nat (sub1 n)))]))
```

Exemplo: soma naturais

Implementação: o caso base.

```
;; Natural -> Natural
;; Soma todos os números naturais de 0 até n
(examples
 (check-equal? (soma-nat 0) 0)
 (check-equal? (soma-nat 1) 1)  ; (+ 1 0)
 (check-equal? (soma-nat 2) 3)  ; (+ 2 1 0)
 (check-equal? (soma-nat 3) 6)) ; (+ 3 2 1 0)
(define (soma-nat n)
  (cond
    [(zero? n) 0]
    [else
      (... n
           (soma-nat (sub1 n)))]))
```

Exemplo: soma naturais

Implementação: o outro caso.

```
;; Natural -> Natural
;; Soma todos os números naturais de 0 até n
(examples
 (check-equal? (soma-nat 0) 0)
 (check-equal? (soma-nat 1) 1)  ; (+ 1 0)
 (check-equal? (soma-nat 2) 3)  ; (+ 2 1 0)
 (check-equal? (soma-nat 3) 6)) ; (+ 3 2 1 0)
(define (soma-nat n)
  (cond
    [(zero? n) 0]
    [else
      (+ n
         (soma-nat (sub1 n)))]))
```

Exemplo: lista de números

Dado um número natural *n*, defina uma função que devolva `(list 1 2 ... n-1 n)`.

Exemplo: lista de números

Especificação

```
;; Natural -> ListaDeNúmeros
;; Cria uma lista com os valores 1 2 ... n-1 n
(examples
  (check-equal? (lista-num 0) empty)
  (check-equal? (lista-num 1) (cons 1 empty))
  (check-equal? (lista-num 2) (cons 1 (cons 2 empty)))
  (check-equal? (lista-num 3) (cons 1 (cons 2 (cons 3 empty)))))
(define (lista-num n) empty)
```

Exemplo: lista de números

Implementação: modelo.

```
;; Natural -> ListaDeNúmeros
;; Cria uma lista com os valores 1 2 ... n-1 n
(examples
  (check-equal? (lista-num 0) empty)
  (check-equal? (lista-num 1) (cons 1 empty))
  (check-equal? (lista-num 2) (cons 1 (cons 2 empty)))
  (check-equal? (lista-num 3) (cons 1 (cons 2 (cons 3 empty)))))
(define (lista-num n)
  (cond
    [(zero? n) ...]
    [else
      (... n
           (lista-num (sub1 n)))]))
```

Exemplo: lista de números

Implementação: caso base.

```
;; Natural -> ListaDeNúmeros
;; Cria uma lista com os valores 1 2 ... n-1 n
(examples
  (check-equal? (lista-num 0) empty)
  (check-equal? (lista-num 1) (cons 1 empty))
  (check-equal? (lista-num 2) (cons 1 (cons 2 empty)))
  (check-equal? (lista-num 3) (cons 1 (cons 2 (cons 3 empty)))))
(define (lista-num n)
  (cond
    [(zero? n) empty]
    [else
      (... n
           (lista-num (sub1 n)))]))
```

Exemplo: lista de números

Implementação: o outro caso.

```
;; Natural -> ListaDeNúmeros
;; Cria uma lista com os valores 1 2 ... n-1 n
(examples
  (check-equal? (lista-num 0) empty)
  (check-equal? (lista-num 1) (cons 1 empty))
  (check-equal? (lista-num 2) (cons 1 (cons 2 empty)))
  (check-equal? (lista-num 3) (cons 1 (cons 2 (cons 3 empty)))))
(define (lista-num n)
  (cond
    [(zero? n) empty]
    [else
      (cons-fim n
                (lista-num (sub1 n)))]))
```

Exemplo: adiciona no final da lista

Especificação para `cons-fim`.

```
;; Número ListaDeNúmeros -> ListaDeNúmeros
;; Adiciona n ao final de ldn.
(define (cons-fim n ldn) ldn)
```

Exemplo: adiciona no final da lista

Especificação para `cons-fim`.

```
;; Número ListaDeNúmeros -> ListaDeNúmeros
;; Adiciona n ao final de ldn.
(examples
  (check-equal? (cons-fim 3 empty)
                (cons 3 empty))
  (check-equal? (cons-fim 4 (cons 3 empty))
                (cons 3 (cons 4 empty)))
  (check-equal? (cons-fim 1 (cons 3 (cons 4 empty)))
                (cons 3 (cons 4 (cons 1 empty)))))
(define (cons-fim n ldn) ldn)
```

Exemplo: adiciona no final da lista

Implementação: modelo.

```
;; Número ListaDeNúmeros -> ListaDeNúmeros
;; Adiciona n ao final de ldn.
(examples
  (check-equal? (cons-fim 3 empty)
                (cons 3 empty))
  (check-equal? (cons-fim 4 (cons 3 empty))
                (cons 3 (cons 4 empty)))
  (check-equal? (cons-fim 1 (cons 3 (cons 4 empty)))
                (cons 3 (cons 4 (cons 1 empty)))))
(define (cons-fim n ldn)
  (cond
    [(empty? ldn) ... n]
    [else
     (... (first ldn)
          (cons-fim n (rest ldn)))]))
```

Exemplo: adiciona no final da lista

Implementação: caso base.

```
;; Número ListaDeNúmeros -> ListaDeNúmeros
;; Adiciona n ao final de ldn.
(examples
  (check-equal? (cons-fim 3 empty)
                (cons 3 empty))
  (check-equal? (cons-fim 4 (cons 3 empty))
                (cons 3 (cons 4 empty)))
  (check-equal? (cons-fim 1 (cons 3 (cons 4 empty)))
                (cons 3 (cons 4 (cons 1 empty)))))
(define (cons-fim n ldn)
  (cond
    [(empty? ldn) (cons n empty)]
    [else
     (... (first ldn)
          (cons-fim n (rest ldn)))]))
```

Exemplo: adiciona no final da lista

Implementação: outro caso.

```
;; Número ListaDeNúmeros -> ListaDeNúmeros
;; Adiciona n ao final de ldn.
(examples
  (check-equal? (cons-fim 3 empty)
                (cons 3 empty))
  (check-equal? (cons-fim 4 (cons 3 empty))
                (cons 3 (cons 4 empty)))
  (check-equal? (cons-fim 1 (cons 3 (cons 4 empty)))
                (cons 3 (cons 4 (cons 1 empty)))))
(define (cons-fim n ldn)
  (cond
    [(empty? ldn) (cons n empty)]
    [else
     (cons (first ldn)
           (cons-fim n (rest ldn)))]))
```

Inteiros

Às vezes queremos utilizar um caso base diferente de 0.

Podemos generalizar a definição de número natural para incluir um limite inferior diferente de 0.

Definição Inteiro

Um número **Inteiro>=a** é

- *a*; ou
- `(add1 n)` onde *n* é um número **Inteiro>=a**

Um número **Inteiro>=a** é

- *a*; ou
- `(add1 n)` onde *n* é um número **Inteiro>=a**

```
(define (fn-para-inteiro>=a n)
  (cond
    [(<= n a) ...]
    [else
      (... n
           (fn-para-inteiro>=a (sub1 n)))]))
```

Limitações

Limitações

Cada tipo com autorreferência tem um modelo de função que podemos usar como ponto de partida para implementar funções que processam o tipo de dado.

Embora o modelo seja um ponto de partida, em algumas situações ele pode não ser útil.

Considere o problema de verificar se uma lista de números é palíndromo (a lista tem os mesmos elementos quando lida da direita para a esquerda e da esquerda para direita).

Para verificar se `(list 5 4 1 4)` é palíndromo, o modelo sugere verificar se `(list 4 1 4)` é palíndromo.

Considere o problema de verificar se uma lista de números é palíndromo (a lista tem os mesmos elementos quando lida da direita para a esquerda e da esquerda para direita).

Para verificar se `(list 5 4 1 4)` é palíndromo, o modelo sugere verificar se `(list 4 1 4)` é palíndromo.

Como a verificação se `(list 4 1 4)` é palíndromo pode nos ajudar a determinar se `(list 5 4 1 4)` é palíndromo?

Considere o problema de verificar se uma lista de números é palíndromo (a lista tem os mesmos elementos quando lida da direita para a esquerda e da esquerda para direita).

Para verificar se `(list 5 4 1 4)` é palíndromo, o modelo sugere verificar se `(list 4 1 4)` é palíndromo.

Como a verificação se `(list 4 1 4)` é palíndromo pode nos ajudar a determinar se `(list 5 4 1 4)` é palíndromo? Ou seja, a solução para o resto pode nós ajustar a compor o resultado para o todo?

Considere o problema de verificar se uma lista de números é palíndromo (a lista tem os mesmos elementos quando lida da direita para a esquerda e da esquerda para direita).

Para verificar se `(list 5 4 1 4)` é palíndromo, o modelo sugere verificar se `(list 4 1 4)` é palíndromo.

Como a verificação se `(list 4 1 4)` é palíndromo pode nos ajudar a determinar se `(list 5 4 1 4)` é palíndromo? Ou seja, a solução para o resto pode nós ajustar a compor o resultado para o todo? Não...

Considere o problema de verificar se um número natural n é primo (tem exatamente dois divisores distintos, 1 e n).

Para verificar se $n = 13$ é primo, o modelo sugere verificar se 12 é primo.

Considere o problema de verificar se um número natural n é primo (tem exatamente dois divisores distintos, 1 e n).

Para verificar se $n = 13$ é primo, o modelo sugere verificar se 12 é primo.

Como a verificação se 12 é primo pode nos ajudar a determinar se 13 é primo?

Considere o problema de verificar se um número natural n é primo (tem exatamente dois divisores distintos, 1 e n).

Para verificar se $n = 13$ é primo, o modelo sugere verificar se 12 é primo.

Como a verificação se 12 é primo pode nos ajudar a determinar se 13 é primo? Não pode...

O problema nos dois casos é o mesmo: a solução do problema original não pode ser obtida a partir da solução do subproblema gerado pela decomposição estrutural do dado.

Como fazemos nesse caso?

O problema nos dois casos é o mesmo: a solução do problema original não pode ser obtida a partir da solução do subproblema gerado pela decomposição estrutural do dado.

Como fazemos nesse caso? Temos algumas opções:

- Redefinimos o problema de forma que a solução para o subproblema estrutural possa ser usado na construção da solução do problema original;
- Fazemos uma decomposição em subproblema(s) de maneira não estrutural e utilizamos a solução desse(s) subproblema(s) para construir a solução do problema original;
- Criamos uma plano (sequência de etapas) para construir a solução sem necessariamente pensar na decomposição da entrada em subproblemas do mesmo tipo.

Para o problema do número primo, podemos rescrever o problema da seguinte forma: Dado dois números naturais n e $a \leq n$, projete uma função que determine a quantidade de divisores de n que são $\leq a$.

Se temos a quantidade de divisores de n que são $\leq a - 1$, como obtemos a quantidade de divisores de n que são $\leq a$?

Para o problema do número primo, podemos rescrever o problema da seguinte forma: Dado dois números naturais n e $a \leq n$, projete uma função que determine a quantidade de divisores de n que são $\leq a$.

Se temos a quantidade de divisores de n que são $\leq a - 1$, como obtemos a quantidade de divisores de n que são $\leq a$? Somando 1 se a é divisor de n.

Como podemos utilizar essa função para determinar se um número n é primo?

Para o problema do número primo, podemos rescrever o problema da seguinte forma: Dado dois números naturais n e $a \leq n$, projete uma função que determine a quantidade de divisores de n que são $\leq a$.

Se temos a quantidade de divisores de n que são $\leq a - 1$, como obtemos a quantidade de divisores de n que são $\leq a$? Somando 1 se a é divisor de n.

Como podemos utilizar essa função para determinar se um número n é primo? Com a expressão `(= (num-divisores n n) 2)`

Número primo

```
;; Natural -> Boolean
;; Produz #t se n é um número primo, isto é, tem exatamente dois divisores distintos (1 e n).
;; Produz #f caso contrário.
(examples
 (check-equal? (primo? 1) #f)
 (check-equal? (primo? 2) #t)
 (check-equal? (primo? 6) #f)
 (check-equal? (primo? 7) #t)
 (check-equal? (primo? 10) #f))
(define (primo? n)
  ;; Natural Natural -> Natural
  ;; Calcula o número de divirores de n que são <= a.
  (define (num-divisores n a)
    (cond
      [(zero? a) 0]
      [(= (remainder n a) 0) (add1 (num-divisores n (sub1 a)))]
      [else (num-divisores n (sub1 a))]))
  (= (num-divisores n n) 2))
```

Para o problema da lista palíndromo, vamos considerar a entrada `(list 4 1 5 1 4)`.

Como podemos obter um subproblema da entrada de maneira que a resposta para o subproblema possa nos ajudar a compor a resposta para o problema original?

Para o problema da lista palíndromo, vamos considerar a entrada `(list 4 1 5 1 4)`.

Como podemos obter um subproblema da entrada de maneira que a resposta para o subproblema possa nos ajudar a compor a resposta para o problema original? Removendo o primeiro e último elemento da lista.

Se sabemos que uma lista `lst` sem o primeiro e o último elemento é palíndromo, como determinar se `lst` é palíndromo?

Para o problema da lista palíndromo, vamos considerar a entrada `(list 4 1 5 1 4)`.

Como podemos obter um subproblema da entrada de maneira que a resposta para o subproblema possa nos ajudar a compor a resposta para o problema original? Removendo o primeiro e último elemento da lista.

Se sabemos que uma lista `lst` sem o primeiro e o último elemento é palíndromo, como determinar se `lst` é palíndromo? Verificando se o primeiro e o último elemento de `lst` são iguais.

Palíndromo 1

```
;; Lista -> Booleano
;; Produz #t se lst é palindromo, isto é, tem os mesmos elementos
;; quando lida da direita para esquerda e da esquerda para direita.
;; Produz #f caso contrário.
(examples
 (check-equal? (palindromo? empty) #t)
 (check-equal? (palindromo? (list 2)) #t)
 (check-equal? (palindromo? (list 1 2)) #f)
 (check-equal? (palindromo? (list 3 3)) #t)
 (check-equal? (palindromo? (list 3 7 3)) #t)
 (check-equal? (palindromo? (list 3 7 3 3)) #f))
(define (palindromo? lst)
  (cond
    [(empty? lst) #t]
    [(empty? (rest lst)) #t]
    [else (and (equal? (first lst) (last lst))
               (palindromo? (sem-extremos lst)))]))
```

Exercício: Implemente a função `sem-extremos` e revise a função `palindromo?`.

Funções recursivas que operam em subproblemas obtidos pela decomposição estrutural dos dados são chamadas de **funções recursivas estruturais**.

Funções recursivas que operam em subproblemas arbitrários (não estruturais) são chamadas de **funções recursivas generativas**.

O projeto de função recursivas generativas pode requerer um "*insight*" e por isso tentamos primeiramente resolver os problemas com recursão estrutural.

Ainda para o problema da lista palíndromo, ao invés de pensarmos em decompor o problema em um subproblema da mesma natureza, podemos pensar em um plano, uma sequencia de etapas que resolva problemas intermediários mas que gerem o resultado que estamos esperando no final.

Por exemplo, podemos primeiramente inverter a lista e depois verificar se a lista de entrada e a lista invertida são iguais.

Note que para este caso precisaríamos projetar duas novas funções. Estas funções poderiam ser implementadas usando recursão estrutural.

Palíndromo 2

```
;; Lista -> Booleano
;; Produz #t se lst é palindromo, isto é, tem os mesmos elementos
;; quando lida da direita pra esquerda e da esquerda para direita.
;; Produz #f caso contrário.
(examples
 (check-equal? (palindromo? empty) #t)
 (check-equal? (palindromo? (list 2)) #t)
 (check-equal? (palindromo? (list 1 2)) #f)
 (check-equal? (palindromo? (list 3 3)) #t)
 (check-equal? (palindromo? (list 3 7 3)) #t)
 (check-equal? (palindromo? (list 3 7 3 3)) #f))
(define (palindromo? lst)
  (equal? lst (reverse lst)))
```

```
;; Lista -> Booleano
;; Produz #t se lst é palindromo, isto é, tem os mesmos elementos
;; quando lida da direita pra esquerda e da esquerda para direita.
;; Produz #f caso contrário.
(examples
 (check-equal? (palindromo? empty) #t)
 (check-equal? (palindromo? (list 2)) #t)
 (check-equal? (palindromo? (list 1 2)) #f)
 (check-equal? (palindromo? (list 3 3)) #t)
 (check-equal? (palindromo? (list 3 7 3)) #t)
 (check-equal? (palindromo? (list 3 7 3 3)) #f))
(define (palindromo? lst)
  (equal? lst (reverse lst)))
```

Note que usamos as funções pré-definidas `equal?` e `reverse`. Como exercício, implemente essas funções.

Revisão

Usamos tipos com autorreferências quando queremos representar dados de tamanhos arbitrários.

- Usamos funções recursivas para processar dados de tipos com autorreferências.

Para ser bem formada, uma definição com autorreferência deve ter:

- Pelo menos um caso base (sem autorreferência): são utilizados para criar os valores iniciais
- Pelo menos um caso com autorreferência: são utilizados para criar novos valores a partir de valores existentes

As vezes é interessante pensar em números inteiros e naturais como sendo compostos e definidos com autorreferência.

Existem dois tipos de recursão: estrutural e generativa.

- A recursão estrutural é aquela feita na decomposição natural do dado (para as partes que são autorreferências na definição do dado).
- A recursão generativa é aquela que não é estrutural.

A recursão estrutural só pode ser utilizada quando solução do problema pode ser expressa em termos da solução do problema estrutural. Para os demais problemas podemos tentar três abordagem:

- Alterar o problema e utilizar recursão natural;
- Usar recursão generativa;
- Usar um plano (sequência de etapas)

Referências

Referências

Básicas

- Vídeos Self-Reference
- Vídeos Naturals
- Capítulos 8 a 12 do livro HTDP
- Seções 2.3, 2.4 e 3.8 do Guia Racket

Complementares

- Seções 2.1 (2.1.1 - 2.1.3) e 2.2 (2.2.1) do livro SICP
- Seções 3.9 da Referência Racket
- Seção 6.3 do livro TSPL4